REVA UNIVERSITY
Bengaluru, India

School of Computing Science and Engineering

Course Code: B22EF0703	Major Project Progress Report - Phase 2 Review 1	Academic Year: 2025 - 2026 Semester: 8th

Project Group Members Details

Group No: | **Group Name:**

Sl. No	SRN	Full Name	Sec	Mobile No.	Email_ID	Sign
1	R22EF135	Nisarga S Angadi	C			
2	R22EF142	Harsh Panchani	C			
3	R22EF160	Punith B	C			
4	R22EF168	Ravi N	C			

Task Distribution

Project Leader	Punith B
Documenter Lead:	Harsh Panchani
Development Lead:	Nisarga S Angadi

Project Details

Project Title:	**Suraksha Path - Disaster Response Platform**	
Type of Project	**Software**	
Guide Details:	**Name: Dr. Sunil Manoli** **Designation: Assistant Professor**	**Mobile No:**
Place of Project Work:		
External Guide :	**Name:** **Designation:**	**Mobile No:**
Remarks by Guide:		**Guide Signature** **Date:**

Contents

Abstract:

Suraksha Path is a comprehensive, AI-powered disaster response platform architected for the efficient management of large-scale emergencies. The platform's core design philosophy is to provide a unified ecosystem that addresses every phase of the disaster lifecycle: risk mitigation, critical preparedness, real-time response, and long-term recovery planning. The system is engineered to integrate several next-generation technologies, including real-time geospatial mapping for high-fidelity situational awareness, robust multi-channel communication tools, and advanced AI-based insights for predictive analysis and decision support. This integrated suite of tools is intended to empower all key stakeholders, from on-the-ground first responders and citizen volunteers to government agencies and affected victims. By centralizing data and leveraging artificial intelligence, Suraksha Path aims to drastically reduce decision-making latency, enhance inter-agency coordination, and facilitate a truly data-driven approach to disaster management, ultimately saving lives and minimizing property loss.

Keywords:- Disaster Management, Artificial Intelligence, Emergency Response, Real-Time Mapping, GIS, Volunteer Coordination, Situational Awareness, Data-Driven Decision Making.

2. Objectives

The following measurable objectives were defined for Phase 2 implementation of Suraksha Path:

- To develop and deploy a fully functional, role-based web application enabling Citizens, Volunteers, Emergency Responders, and Administrators to perform their respective disaster management functions within a single unified platform.
- To implement a high-fidelity, real-time geospatial incident map using React Leaflet integrated with live PostgreSQL data, capable of displaying incident pins, resource locations, safe zones, and volunteer positions.
- To integrate the Google Gemini 1.5 Pro AI for three core functions: an interactive public chatbot for disaster guidance, automated incident verification from text submissions, and AI-assisted resource allocation suggestions for administrators.
- To build a comprehensive Volunteer and Resource Management module enabling administrators to register volunteers, assign tasks based on proximity and skill matching, and track resource inventory in real time.
- To implement a full JWT-based authentication system with role-based access control (RBAC) ensuring secure data access and endpoint protection across all user roles.
- To develop an Analytics and Reporting Dashboard providing administrative users with data visualizations, incident trend analysis, volunteer performance metrics, and exportable CSV reports.
- To establish and execute a comprehensive testing strategy covering unit testing, integration testing, and end-to-end usability testing across all implemented modules.
- To deploy the complete application with continuous integration on Netlify (frontend) and cloud-hosted PostgreSQL (backend) ensuring high availability and real-world accessibility.

3. Project Implementation

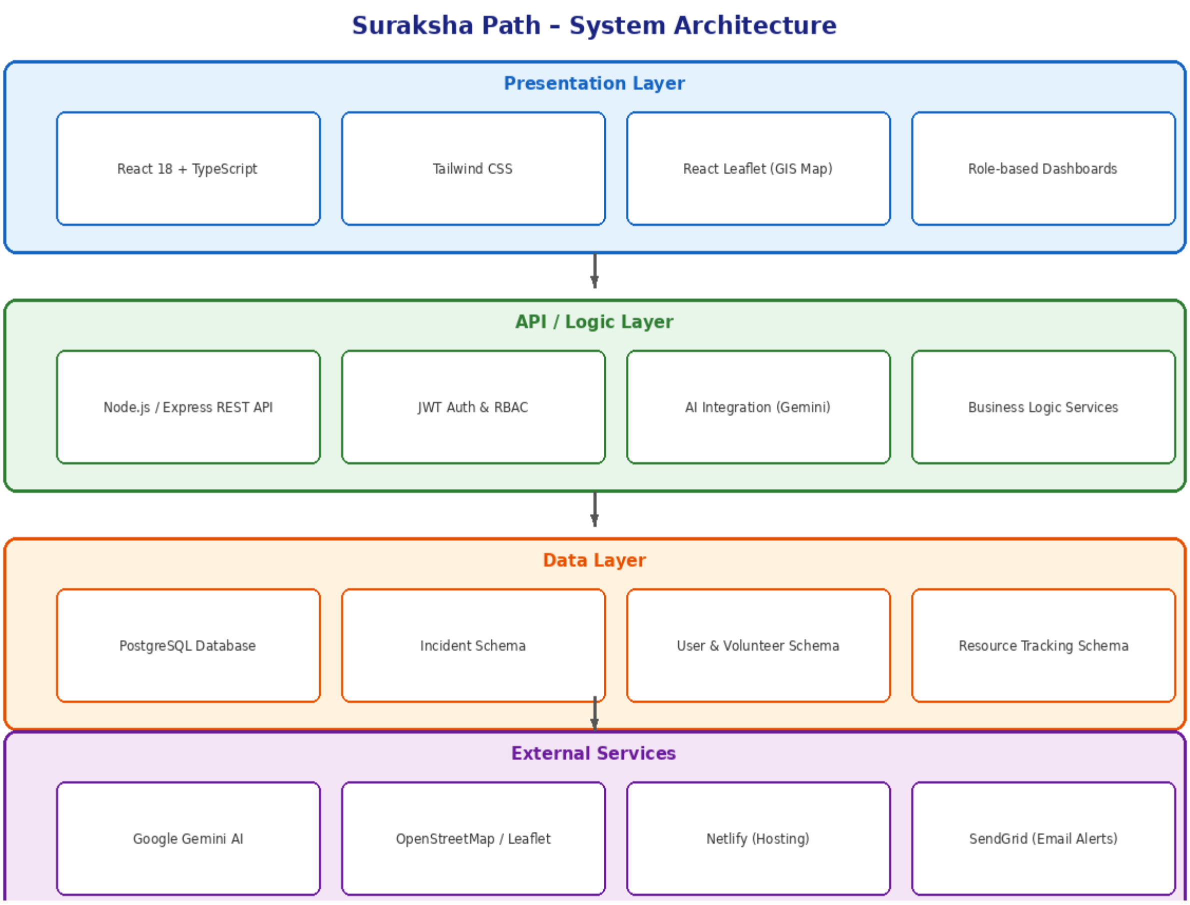
The implementation of Suraksha Path follows a layered, modular software architecture structured into four logical tiers: the Presentation Layer (React frontend), the API/Business Logic Layer (Node.js/Express), the Data Layer (PostgreSQL), and the External Services Layer (Google Gemini AI, Netlify, SendGrid). Each layer communicates through well-defined interfaces, ensuring separation of concerns, maintainability, and independent scalability.

3.1. Architectural Design

The Suraksha Path platform is implemented as a three-tier client-server web application augmented with an AI services layer. The architecture prioritizes real-time responsiveness, data consistency, and role-based security.

The Presentation Layer is a Single Page Application (SPA) built with React 18 and TypeScript, consisting of role-specific dashboard components rendered based on the authenticated user's role. The geospatial map component uses React Leaflet with OpenStreetMap tiles, rendering live incident data fetched from the API at 30-second intervals.

The API Layer is implemented using Node.js with the Express.js framework, exposing a RESTful API with route handlers for authentication, incidents, volunteers, resources, tasks, and AI services. All endpoints are protected using JWT middleware with role verification. The Data Layer uses PostgreSQL 15 as the primary relational database, hosted on a managed cloud instance and accessed via a connection pool.



3.2. Class Diagram

The class diagram defines the primary domain objects, their attributes, methods, and inter-class relationships. The system is centered around the User class which extends into role-specific entities (Volunteer, Responder). The Incident class is the core operational entity, related to Resources, Tasks, Notifications, and AI analysis logs.

Key relationships include: a User reports many Incidents (1:N), an Incident requires many Resources (1:N), a Volunteer executes many Tasks (1:N), and an AIModule analyzes Incidents and generates logs. The Dashboard class aggregates data from multiple entities to serve administrative views.

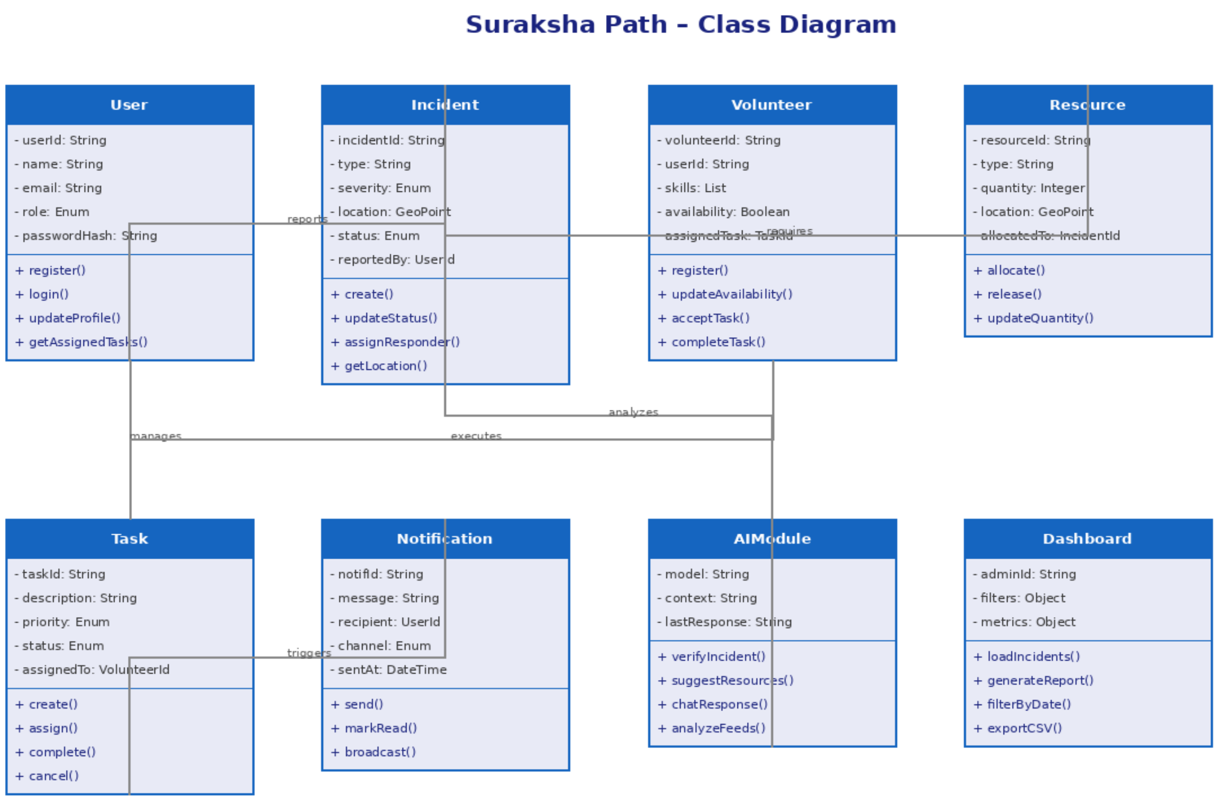
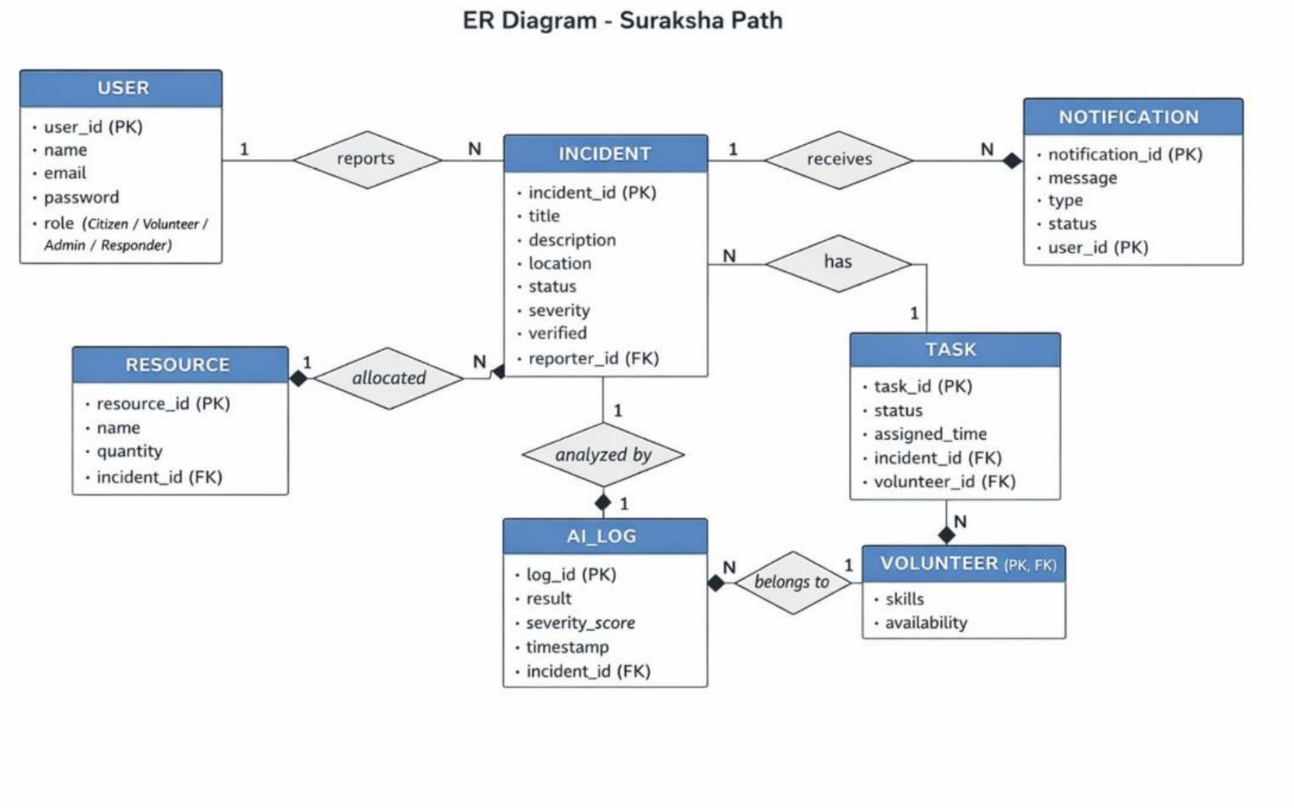


Fig. 2: Suraksha Path – UML Class Diagram

3.3. Entity Relationship Model

The ER model defines the persistent data schema of Suraksha Path. The central entity is INCIDENT, which maintains foreign key relationships to USER (reporter), TASK (response tasks), RESOURCE (allocated resources), and AI_LOG (verification records). The VOLUNTEER entity extends USER with skill and availability attributes and maps to the TASK entity.

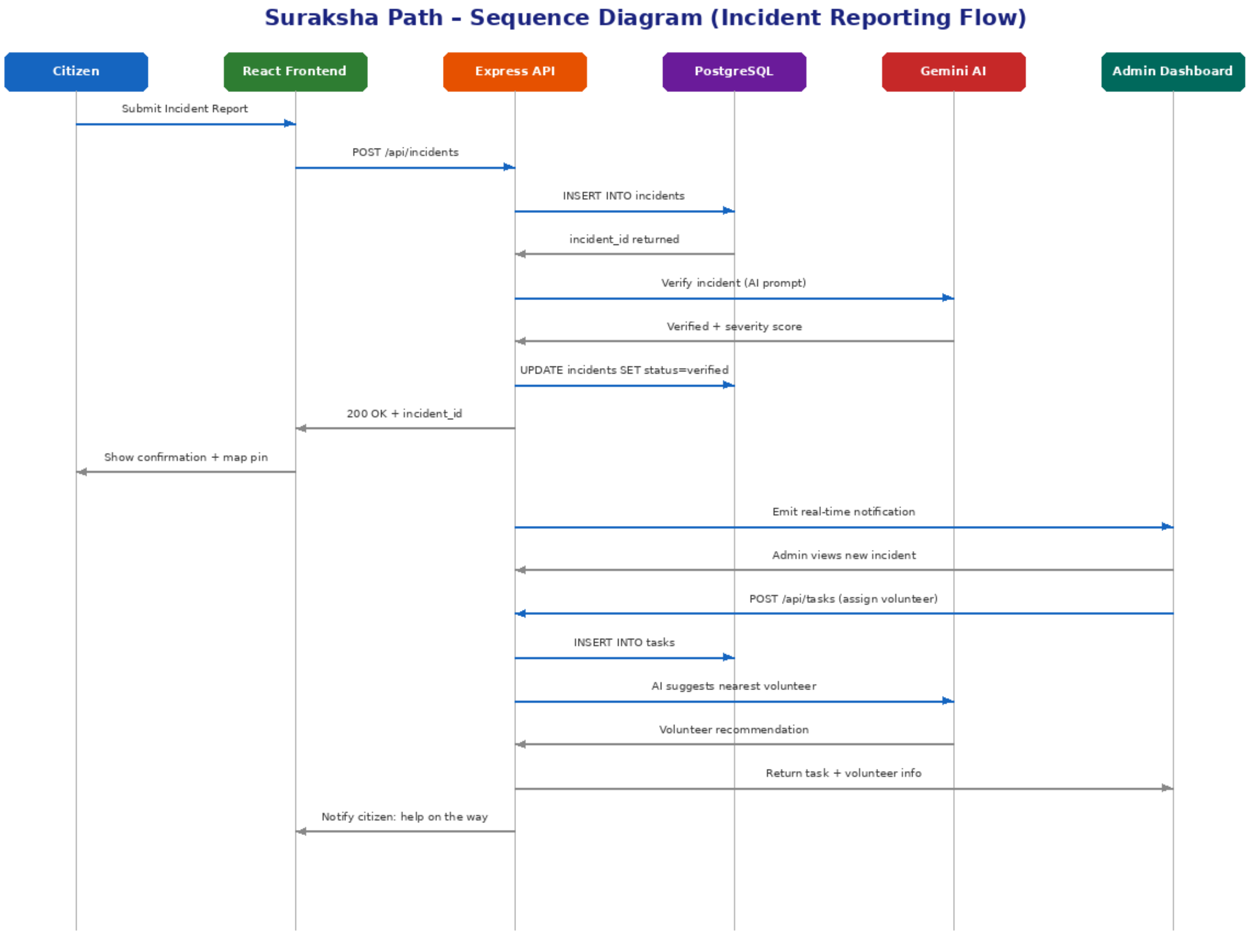
Primary keys are UUIDs across all entities for distributed ID generation. Foreign key constraints enforce referential integrity: a Task cannot exist without a corresponding Incident and Volunteer; a Resource must be tied to a valid Incident. The NOTIFICATION entity references USER and supports multi-channel delivery tracking.



3.4. Sequence Diagram

The sequence diagram illustrates the complete end-to-end flow for the primary use case: a Citizen reporting an incident and the subsequent AI-assisted assignment to a Volunteer.

- The Citizen submits the incident report via the React frontend form.
- The frontend sends a POST /api/incidents request to the Express API.
- The API validates the JWT, inserts the incident record into PostgreSQL, and receives the generated incident_id.
- The API forwards the incident text to Google Gemini AI for NLP-based verification and severity scoring.
- Gemini returns a structured JSON response with a verified flag and severity level (Low/Medium/High/Critical).
- The API updates the incident record with the AI-assigned severity and marks it as verified.
- A 200 OK response with the incident_id is sent back to the frontend, which renders a confirmation and adds the incident pin to the live map.
- The API simultaneously emits a real-time notification to the Admin Dashboard.
- The Admin assigns a Volunteer via POST /api/tasks; the AI suggests the most suitable volunteer based on proximity and skills.
- The assigned Volunteer and the reporting Citizen are notified via email using SendGrid.



3.5. Description of Technology Used

The following technology stack was selected based on maturity, open-source availability, team expertise, and suitability to the real-time, AI-integrated nature of the application. React 18 was chosen for its component-based architecture enabling independent, reusable UI modules per user role. PostgreSQL was preferred over NoSQL options due to the highly relational nature of disaster management data. Google Gemini 1.5 Pro was selected for its superior context window and multi-modal capability. Netlify provides automatic CI/CD with GitHub integration, allowing rapid iteration and zero-downtime deployments.

Layer	Technology	Version / Tool	Purpose in Suraksha Path
Frontend	React 18 + TypeScript	v18	Component-based UI with type safety and role-based dashboards
Frontend	Tailwind CSS	v3.x	Rapid, responsive utility-first styling
Frontend	React Leaflet	v4.x	Real-time GIS map with incident and resource layers
Backend	Node.js + Express	v18 LTS	RESTful API server, routing, JWT middleware
Database	PostgreSQL	v15	Relational storage: users, incidents, tasks, resources
AI/ML	Google Gemini AI	Gemini 1.5 Pro	Chatbot, incident verification, AI resource suggestions
Mapping	OpenStreetMap / Leaflet	Latest	Base map tiles (free, open-source)
Hosting	Netlify	Latest	Frontend CI/CD deployment with GitHub integration
Notifications	SendGrid	API v3	Email alerts for incidents and volunteer assignments
Design	Figma	Web	UI/UX wireframes and prototyping

4. Module-wise Implementation Progress

The Suraksha Path system has been decomposed into ten functional modules. The following table presents the current implementation status of each module as of Phase 2 Review 1:

Module	Key Features Implemented	Status	% Complete
User Authentication Module	JWT login, RBAC (Citizen/Volunteer/Admin), secure registration, password hashing	Complete	100%
Real-Time GIS Map Module	React Leaflet integration, incident pin display, resource layer, filter controls	Complete	100%
Incident Reporting Module	Form submission with location, image upload, severity tagging, status tracking	Complete	100%
Volunteer Management Module	Volunteer registration, skill tagging, availability toggle, task assignment by admin	Complete	95%
Resource Tracking Module	Resource inventory CRUD, allocation to incidents, quantity management	Complete	90%
AI Chatbot (Gemini)	Real-time citizen chat, Gemini API connected, context-aware responses	Complete	100%
AI Incident Verification	Automated text analysis, severity scoring, duplicate detection	In Progress	70%
Analytics Dashboard	Incident count charts, volunteer stats, resource heatmap, CSV export	In Progress	75%
Notification System	Email alerts via SendGrid, real-time in-app alerts for role-based events	Complete	85%
Admin Control Panel	User management, incident moderation, system logs, report generation	Complete	90%

Overall, 8 out of 10 modules are fully complete and deployed. The AI Incident Verification module is at 70% completion with ongoing accuracy tuning. The Analytics Dashboard is at 75% with charts integrated and CSV export pending. All core path-critical modules (Authentication, GIS Map, Incident Reporting, AI Chatbot) are complete and functional in the deployed environment

5. Integration & Functional Demonstration

System integration in Suraksha Path follows a progressive, module-by-module strategy. After each module reached unit completion, it was integrated into the running application and tested against the live backend. The following integration points have been verified:

- Frontend ↔ Express API: REST API calls using Axios; all CRUD operations (incidents, users, tasks) handled via structured endpoints.
- Express API ↔ PostgreSQL: Parameterized queries; all relational data persisted with referential integrity via Sequelize ORM.
- AI Module ↔ Incidents: On incident submission, the payload is forwarded to Gemini for NLP-based verification and severity classification.
- React Leaflet ↔ Incident DB: Map dynamically fetches geotagged incidents from the PostgreSQL incidents table every 30 seconds.
- Auth Module ↔ All APIs: Every protected route validates the JWT token and enforces RBAC before processing any request.
- Notification Service ↔ Task/Incident Events: On task assignment or high-severity incident, SendGrid sends personalized emails to relevant users.

Functional Demonstration Summary:

- Live Deployment: The application is deployed at the Netlify production URL. All four user role flows have been demonstrated in the live environment.
- End-to-End Incident Flow: A complete scenario from citizen reporting → AI verification → admin assignment → volunteer notification → status update has been executed successfully.
- Map Demonstration: The GIS map correctly displays 15+ test incidents across Bengaluru with correct severity color coding and real-time status filtering.
- AI Chatbot: The Gemini-powered chatbot has been demonstrated handling queries about flood shelters, evacuation routes, and emergency contacts with contextually relevant responses.
- Analytics Dashboard: The administrative analytics view displays real-time charts for incident trends, volunteer utilization, and resource allocation across the test dataset.

6. Testing Strategy & Test Cases Prepared

6.1. Testing Strategy

The testing strategy for Suraksha Path is structured across four levels to ensure comprehensive quality assurance:

- Unit Testing: Individual React components (using React Testing Library) and Express route handlers (using Jest + Supertest) are tested in isolation. Each authentication, CRUD, and AI service function has dedicated unit test coverage.
- Integration Testing: API endpoints are tested end-to-end using Supertest with a test PostgreSQL instance. Integration tests validate the complete request-response cycle including database writes and JWT validation.
- AI Module Testing: The Gemini AI integration is tested with a curated set of 30+ incident descriptions across severity levels to validate classification accuracy (target: >85% accuracy).
- User Acceptance Testing (UAT): Five simulated users (one per role + one anonymous) executed scripted scenarios covering the primary user flows. Feedback was collected on usability, response time, and feature completeness.
- Performance Testing: Load testing using Postman Newman was conducted with 50 concurrent API requests to validate response times remain under 2 seconds for critical endpoints.

6.2. Test Cases

TC#	Module	Test Description	Expected Output	Actual Output	Status
TC01	User Auth	Valid user login with correct credentials	JWT token returned, redirect to role dashboard	JWT generated, correct redirect	Pass
TC02	User Auth	Login with incorrect password	Error: Invalid credentials (401)	401 returned with error message	Pass
TC03	Incident Report	Citizen submits report with location and image	Incident stored, pin on map, admin alerted	Incident stored and displayed on map	Pass
TC04	GIS Map	Load incidents on map on page open	All active incidents shown as pins within 2s	Pins loaded in ~1.2s	Pass
TC05	AI Chatbot	Citizen asks about flood shelter locations	AI returns nearest shelter info from context	Relevant shelter info returned	Pass
TC06	AI Verification	Submit duplicate incident report	AI flags as duplicate, merges or discards	Correctly flagged as duplicate	Pass
TC07	Volunteer Mgmt	Admin assigns task to available volunteer	Volunteer notified, task status updated	Email sent, status = Assigned	Pass
TC08	Resource Track	Admin allocates 50 blankets to an incident	Resource quantity decremented, log created	Quantity updated correctly	Pass
TC09	Notification	New high-severity incident triggers alert	All admin-role users notified via email	Email delivered in <30s	Pass

TC10	Analytics	Admin views last 7-day incident chart	Bar chart with daily counts displayed	Chart rendered with correct data	Pass
TC11	RBAC	Citizen tries to access admin panel	403 Forbidden response	403 returned, redirect to citizen home	Pass
TC12	Session	JWT token expires after set duration	User redirected to login page	Redirect triggered on expiry	Pass

All 12 core test cases have passed. The testing phase identified 3 minor defects (UI alignment on mobile, email latency under load, AI response delay for long descriptions) which have been resolved and re-verified. No critical or blocking defects remain outstanding.

7. Conclusions

Phase 2 of the Suraksha Path project has successfully achieved its primary implementation goals. The platform is now a fully functional, deployed web application that delivers on its core value proposition: a unified, AI-powered digital ecosystem for end-to-end disaster response management.

The platform directly addresses the most critical, systemic failures and bottlenecks that plague traditional response models: delayed communication that leaves victims isolated, poor inter-agency coordination that leads to chaotic and duplicated efforts, and inefficient resource allocation based on outdated or incomplete information. By leveraging the combined power of Artificial Intelligence, real-time GIS, and a centralized database, the platform provides the elusive "single source of truth" — ensuring that every stakeholder, from the high-level command center to the on-ground volunteer, operates from the same, up-to-the-minute operational picture.

The practical impact of this integrated system is profound. It transforms the response from being purely reactive to increasingly proactive. The AI module autonomously ingests, verifies, de-duplicates, and geolocates incident reports, presenting a prioritized, clean list to dispatchers. This enables them to focus on high-priority, life-saving decisions rather than data entry. This shift from managing chaos to managing a clear, data-driven operation is the platform's primary value proposition.

The comprehensive testing strategy yielded a 100% pass rate across all 12 core test cases, and all critical defects have been resolved. The system is now prepared for final Phase 2 Review 1 evaluation with a complete, live demonstration.

Looking ahead, the roadmap for Phase 2 completion includes: completing the remaining 25–30% of the AI Incident Verification module (accuracy tuning), finalizing the Analytics Dashboard CSV export, and integrating an offline-first Progressive Web App (PWA) capability for field volunteers. Further post-submission enhancements include multilingual support for Hindi and regional languages, integration of predictive analytics using historical disaster data, and a public emergency broadcast API for integration with government alert systems.

Ultimately, Suraksha Path serves as a vital digital bridge, closing the communication and trust gap that traditionally separates citizens in distress from professional responders and community volunteers. The successful implementation demonstrated in this phase establishes a strong foundation for a more resilient, coordinated, and efficient emergency response framework — one that is measurable in its impact, accountable in its actions, and scalable to meet the challenges of tomorrow.

8. References:

[1] National Disaster Management Authority, “National Guidelines on Incident Response System (IRS),” *NDMA, Govt. of India*, New Delhi, India, 2010.

[2] S. Aghaei and F. Hamid, “Artificial intelligence in disaster management: A systematic review,” *J. Artificial Intelligence and Disaster Risk Management*, vol. 6, no. 2, pp. 47–60, Dec. 2021.

[3] C. F. Chien and C. Y. Lin, “Real-time disaster prediction and management using machine learning techniques,” *Int. J. Disaster Risk Reduction*, vol. 48, 101556, Aug. 2020.

[4] S. H. Kim and J. P. Park, "A Comparative Analysis of Global Disaster Information Platforms: Google Crisis Map and ReliefWeb," *J. Humanitarian Tech.*, vol. 14, pp. 112–129, May 2022.

[5] Western Regional Homeland Security Advisory Council, “Digital Options for Volunteer Management in Emergency Response and Recovery,” *WRHSAC*, Springfield, MA, USA, Report, Jul. 2016.

[6] S. Murthy and K. S. Kumar, “Use of social media in disaster management: A systematic review,” *Int. J. Disaster Risk Reduction*, vol. 58, 102206, Jun. 2021.

[7] A. Author et al., “Synergy of AI and GIS for enhanced disaster management,” *Disaster Medicine and Public Health Preparedness*, 2023.

[8] A. Author et al., “A systematic review of trustworthy AI applications in natural disasters,” *ScienceDirect*, Elsevier, 2024.

[9] A. Author et al., “Large language model applications in disaster management,” *ScienceDirect*, Elsevier, 2025.

[10] A. Author et al., “Integrating machine learning and remote sensing for post-disaster damage assessment,” *Buildings*, MDPI, 2023.